

Spring Boot Bucket4j Cheat Sheet

Bucket4j or Resilience4j?

Need	Use
Stay under a dependency's limit, per instance	R4j RateLimiter
One 3rd-party quota for all replicas	Bucket4j distributed bucket
Per-client quotas (plan per API key)	Bucket4j : filter or starter
Limits at the edge	Gateway Bucket4j / Redis limiter
Limit concurrency, not rate	Bulkhead , @ConcurrencyLimit
Timeouts, breakers, retries	R4j (Bucket4j has none)

Use both: Bucket4j decides inbound; R4j guards outbound calls and the bucket store (CircuitBreaker + TimeLimiter → fail open or closed); R4j Retry honours Retry-After. R4j = fixed cycles (up to 2x the limit across a boundary); Bucket4j = token bucket (burst = capacity).

Dependencies

Group com.bucket4j, package io.github.bucket4j. Boot does not manage Bucket4j: import the BOM (new in 8.21.0).

```
bucket4j-jdk17-bom:8.21.0 (import)
bucket4j-jdk17-core
bucket4j-jdk17-redis-common
+ -lettuce (or -redisson, -jedis, -vertx, -glide)
bucket4j-jdk17-postgresql (-mysql..)
bucket4j-jdk17-caffeine (local)
```

bucket4j-core to 8.10.1 → renamed _jdkN in 8.11.1; _jdk11 stops at 8.15.0; Java 8 after 8.10.1 is commercial. No 9.x yet.

Starter	Boot	Artifacts
0.13.0	3.5	starter + context
0.14.0	4.0	starter + context
0.20.0-RC1 (no GA)	4.1	starter-servlet -webflux, -aop, one cache-*

Group com.giffing.bucket4j.spring.boot.starter (community). Gateway 5.0.3 has Bucket4j as an optional dependency: declare core + backend yourself.

Token bucket & Bandwidth

tokens(t) = min(C, tokens(t0) + r(t - t0)); wait = (n - k) / r; burst = C.

```
Bucket.builder()
.addLimit(l -> l.capacity(10)
.refillGreedy(10, ofSeconds(1)))
.addLimit(l -> l.capacity(100)
.refillGreedy(100, ofHours(1)))
.build(); // all must allow
```

- refillGreedy (default choice), refillIntervally, refillIntervallyAligned(n, p, first), ...WithAdaptiveInitialTokens (aligned: wall clock only).
- initialTokens(n) (default C), id("..."). Long periods are quotas: add a short burst bandwidth.
- Max 1 token/ns; capacity > 0. **Deprecated:** Bandwidth.simple/classic, Refill.

Bucket API one-liners

```
b.tryConsume(1) // boolean
var p =
b.tryConsumeAndReturnRemaining(5);
p.isConsumed()
p.getRemainingTokens()
p.getNanosToWaitForRefill() // wait
p.estimateAbilityToConsume(n)
b.tryConsumeAsMuchAsPossible()
b.asBlocking().consume(1) // parks
b.asBlocking().tryConsume(1, maxWait)
b.asScheduler()
b.tryConsume(1, maxWait, executor)
b.asVerbose().tryConsume(1) // + state
b.addTokens(n) // capped at C
b.forceAddTokens(n) // may exceed C
b.reset(); b.getAvailableTokens();
```

No Bucket.consume(). Weighted: tryConsume(5). Local builder: withSynchronizationStrategy (LOCK_FREE, SYNCHRONIZED, NONE), millisecond clock by default, withNanosecondPrecision, withCustomTimePrecision, withListener. BucketListener: onConsumed/onRejected (tokens), onParked, onDelayed, onInterrupted.

TokensInheritanceStrategy

```
b.replaceConfiguration(cfg, strategy); C
100 with 40 left, new C 200 / 20:
```

Strategy	Tokens	200 / 20
PROPORTIONALLY	before x newC / oldC	80 / 8
AS_IS	min(before, newC)	40 / 20
ADDITIVE	AS_IS + max(0, newC - oldC)	140 / 20
RESET	as newly built	200 / 20

Bandwidths match by id: same ids (or none) on both sides. Stored buckets: withImplicitConfigurationReplacement(version, strategy).

Distributed: ProxyManager

```
@Bean ProxyManager<String>
pm(RedisClient c) {
var ttl = ExpirationAfterWriteStrategy
.basedOnTimeForRefillingBucketUpToMax(
ofMinutes(10));
return Bucket4jLettuce
.casBasedBuilder(c)
.expirationAfterWrite(ttl)
.requestTimeout(ofMillis(250))
.build()
.withMapper(k -> k.getBytes(UTF_8));
}
BucketProxy b = pm.builder()
.build("plan:" + key, plan::config);
```

Boot creates no RedisClient bean: define it. Lazy: stored on first call. asAsync() → AsyncProxyManager.

Builder settings (no withClientSideConfig): requestTimeout (none), maxRetries (unset = CAS retries forever), retryStrategy (RetryStrategies.exponentialBackoffWithJitter(base, max, budget)), clientClock, expirationAfterWrite (none, fixedTimeToLive, basedOnTimeForRefillingBucketUpToMax), backwardCompatibleWith, defaultListener, defaultRecoveryStrategy (RECONSTRUCT or THROW_BUCKET_NOT_FOUND_EXCEPTION), executionStrategy. build(key, config) without a supplier is deprecated.

Optimizations: batching(), delaying, predicting, skip-on-zero, manual sync; build the proxy once and reuse it. Clock: CAS/lock/JDBC use the app JVM clock; grids use the server's.

Backends (bucket4j-jdk17-*)

Backend	Builder	Atomicity
Redis: Lettuce, Redisson, Jedis, Vert.x, GLIDE	casBasedBuilder	CAS (Lua)
PostgreSQL	selectForUpdateBasedBuilder, advisoryLockBasedBuilder	row / advisory lock
MySQL, MariaDB, Oracle, MSSQL, DB2	selectForUpdateBasedBuilder	row lock
JCache, Coherence	entryProcessorBasedBuilder	entry processor
Hazelcast	entry processor, lock, CAS builders	EP / lock / CAS
Ignite 2 / 3, Infinispan, Geode	thick/thin client, builder(), EP / Hot Rod, CAS / function	EP / server job / CAS
MongoDB, Couchbase, Memcached, Cassandra	compareAndSwapBasedBuilder (Memcached, casBasedBuilder)	CAS
Caffeine (local)	builderFor(...)	local CAS

Entry class: Bucket4jLettuce, Bucket4jPostgreSQL, ... JDBC: table bucket(id, state, expires_at) via Flyway, no async, call removeExpired(n). Per consumed request: Lettuce 2 round trips (GET + EVAL), rejected 1; JDBC 1 transaction; entry processor 1.

Manual Spring Boot filter · 429

```
// OncePerRequestFilter
// .doFilterInternal(req, res, chain)
var p = pm.builder()
.build(key(req), plan(req)::config)
.tryConsumeAndReturnRemaining(
cost(req)); // search: 5
if (!p.isConsumed()) {
// hand to the @ExceptionHandler
resolver.resolveException(req, res, null,
new RateLimitExceededException(p));
return;
}
res.setHeader(
"X-Rate-Limit-Remaining",
String.valueOf(
p.getRemainingTokens()));
chain.doFilter(req, res);
```

HTTP/1.1 429
Retry-After: 3 # ceil(nanos), ≥ 1
X-Rate-Limit-Remaining: 0
Content-Type: application/problem+json
{"title": "Rate limit exceeded",
"status": 429, "detail": "...",
"plan": "free", "retryAfterSeconds": 3}

Build the 429 with ProblemDetail.forStatusAndDetail(TOO_MANY_REQUESTS, ...) in a @RestControllerAdvice (RFC 6585, RFC 9110 Retry-After, RFC 9457).

- Security chain at spring.security.filter.order -100: after it (-99) the key can be the principal; before it (-101) failed logins count too.
- HandlerInterceptor runs after Security, knows the handler; WebFlux: WebFilter + AsyncProxyManager.
- Key by API key or principal; IP only behind trusted proxies; never a spoofable header.
- Store down: requestTimeout + R4j CircuitBreaker → fail closed (503 + Retry-After) or fail open (200). Virtual threads: no asBlocking() on servlet threads anyway.

Starter YAML & @RateLimiting

```
bucket4j:
enabled: true # default
cache-to-use: jcache
filters:
- cache-name: buckets # default
url: /books.* # default .*
strategy: first # or all
rate-limits:
- cache-key: >-
getHeader('X-API-Key')
bandwidths:
- capacity: 20
time: 1
unit: minutes
methods:
- name: isbn
cache-name: buckets
rate-limit:
bandwidths: [...] # singular
```

@RateLimiting(name = "isbn", cacheKey = "#isbn", fallbackMethodName = "..."), @IgnoreRateLimiting (starter-aop + spring-boot-starter-aspectj).

- A community project, recommended by Bucket4j's README.
- filter-order default HIGHEST_PRECEDENCE + 10: before Spring Security, cache-key default "1": one bucket for all.
- 429 with X-Rate-Limit-Retry-After-Seconds, body { "message": "Too many requests!" }; no Retry-After or ProblemDetail.
- cache-to-use: jcache, redis-jedis, redis-lettuce (webflux only, needs a RedisClient bean), redis-redisson, hazelcast*, ignite, infinispan, postgresql (0.20). Caffeine via jcache.
- Meters bucket4j_summary_consumed, _rejected, ...; /actuator/bucket4j. Test with RANDOM_PORT.

Spring Cloud Gateway 5.0.3

```
// Web MVC; needs an
// AsyncProxyManager bean
.filter(rateLimit(c -> c
.setCapacity(3)
.setPeriod(ofMinutes(1))
.setKeyResolver(r -> r.headers()
.firstHeader("X-API-Key"))))
```

Defaults: 1 token, 429, header X-RateLimit-Remaining; missing key → 403; no Retry-After; Java only.

```
# WebFlux RequestRateLimiter args
key-resolver: "#{@apiKeyResolver}"
bucket4j-rate-limiter.capacity: 3
bucket4j-rate-limiter.refillPeriod: 1m
```

Also refillTokens, requestedTokens, refillStyle (GREEDY, INTERVALLY, INTERVALLY_ALIGNED), headerName. Store error → denied (fails closed). Empty keys: spring.cloud.gateway.server.webflux.filter.request-rate-limiter.deny-empty-key (the docs show an old prefix).

Metrics · testing

- No built-in meters: a BucketListener per plan feeds Micrometer (tokens, not calls); defaultListener on the manager, replaced by withListener. Add request-level meters (decision, remaining, store errors); alert on rejection ratio and store errors; add a store HealthIndicator (keep it out of liveness).
- A custom TimeMeter implements currentTimeNanos() and isWallClockBased(); plug it in with withCustomTimePrecision or clientClock.
- N threads admit exactly C; MockMvc / WebTestClient on 429 and headers; tiny bandwidths in a test profile; Testcontainers Redis/PostgreSQL; Toxiproxy outage for fail open/closed.

Anti-patterns · checklist

- asBlocking() on servlet threads; new proxies per request.
- Unbounded keys without expiry; spoofable keys.
- Only after auth (brute force passes); unsynced clocks.
- Per-instance limits sold as global; no outage plan.
- Checklist: handle store exceptions; back up and retain state; HA store; store load = checks/s x round trips; clean expired JDBC rows; login limits per user + address and per address.